

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Problem Solving Assignment

**COURSE NAME: ARTIFICIAL
INTELLIGENCE AND
EXPERT SYSTEMS**

**COURSE OUTCOME COVERED
COURSE CODE: MLA0104**

CO2: *Analyse and select appropriate agent strategies for different types of environments and problem settings. (BTL 3)*

Assessment Tool Weightage: 40 %

**Total Marks: 100
No. of Questions: 5**

Time: 60 Mins

Annexure A Problem Solving Assignment

Code of Conduct:

*I **M. Sanjay Krishna / 192525421** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

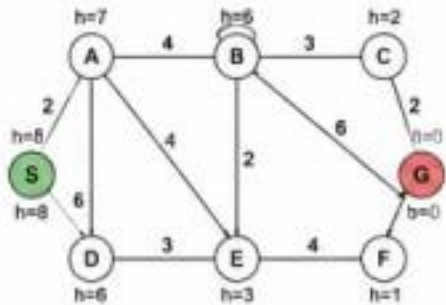
Signature of the student: M. Sanjay

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation Total	10% 100%	

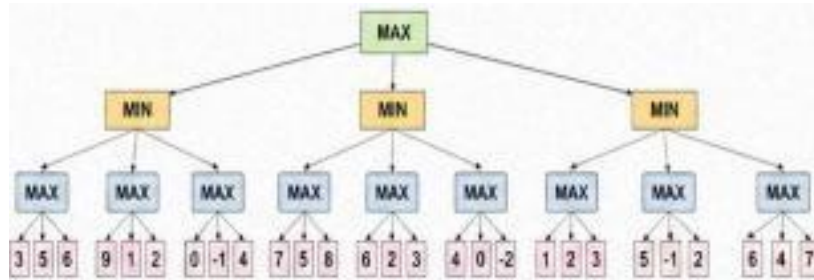
Signature of the Faculty

Total Marks Awarded

Answer All the Questions

Q.No	Application-Based Question	Major CO2 Concepts
1	Smart Ambulance Navigation: An ambulance must travel from a hospital to an emergency location through a city road network. Some roads have higher travel costs, and an estimated distance to the destination is available for each junction. Design a search-based solution to identify the best route. Using the given road network and heuristic values, determine the route selected by your algorithm, show the sequence of junctions explored, calculate the relevant evaluation values, and justify whether the route is optimal. 	Informed Search, A*, Heuristic Function
2	Autonomous Warehouse Robot: A robot has to move from a charging station to a target shelf in aware house. The robot does not know all obstacles initially and discovers some obstacles only when it reaches nearby locations. Design an intelligent search strategy for the robot. Demonstrate the robot's decisions for the given grid, explain how it reacts when a new obstacle is discovered, and compare the approach with an offline search strategy. 	Online Search Agents, Inform Search
3	University Examination Scheduling: A university needs to schedule six examinations (E1, E2, E3, E4, E5, and E6) into four available time slots (T1, T2, T3, and T4). Some examinations have common students and therefore cannot be conducted in the same time slot.	CSP, MRV, Forward Checking, Constraints

	The following constraints must be satisfied: 1. E1 and E2 have common students. 2. E1 and E3 have common students. 3. E2 and E4 have common students. 4. E3 and E5 have common students. 5. E4 and E6 have common students. 6. E2 and E5 have common students. The university has three examination halls with the following capacities:																			
	<table><tr><th>Examination Hall</th><th>Capacity</th></tr><tr><td>Hall A</td><td>120 students</td></tr><tr><td>Hall B</td><td>90 students</td></tr><tr><td>Hall C</td><td>60 students</td></tr></table>	Examination Hall	Capacity	Hall A	120 students	Hall B	90 students	Hall C	60 students											
Examination Hall	Capacity																			
Hall A	120 students																			
Hall B	90 students																			
Hall C	60 students																			
	Examination <table><tr><td>E1</td><td>E2</td><td>E3</td><td>E4</td><td>E5</td><td>E6</td></tr><tr><td>Number of</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>Students</td><td>100</td><td>80</td><td>70</td><td>60</td><td>90</td></tr></table> Tasks: a) Formulate the problem as a Constraint Satisfaction Problem (CSP) by clearly defining the variables, domains, and constraints. b) Use Backtracking with the Minimum Remaining Values (MRV) heuristic and Forward Checking to obtain a valid examination timetable. c) Show the domain of each variable after every assignment and provide the final timetable, ensuring that all student-conflict and hall-capacity constraints are satisfied.	E1	E2	E3	E4	E5	E6	Number of						Students	100	80	70	60	90	
E1	E2	E3	E4	E5	E6															
Number of																				
Students	100	80	70	60	90															
4	Game-Playing AI: You are developing an AI opponent for a board game. The current game position produces the following possible moves and terminal scores in the supplied game tree. Determine the move the AI should make by applying Minimax. Then apply Alpha-Beta Pruning and identify the branches that do not need to be evaluated. Explain how pruning improves the efficiency of the game-playing agent.	Game Playing, Minimax, Alpha-Beta Pruning																		



5

Delivery Route Optimization:

A logistics company wants to minimize the total distance travelled by a delivery vehicle visiting multiple customers. An initial route and several neighbouring routes are provided. Design an optimization strategy to improve the route. Apply the strategy step-by-step to the given routes, determine the final route, explain whether it represents a global optimum, and suggest how the algorithm could escape a local optimum.



Local Search, Optimization, Hill Climbing

Q1. Smart Ambulance Navigation – A* Search

1. Problem Analysis

The ambulance must travel from **S (Start)** to **G (Goal)** through a road network with different travel costs. Heuristic values are provided for the junctions. Hence, **A*** is suitable.

2. Algorithm Selected

A* Search

A* uses: $f(n)=g(n)+h(n)$

where:

- $g(n)$ = cost from S to current node
- $h(n)$ = heuristic cost to G
- $f(n)$ = total evaluation value

3. Given Values

Node	$h(n)$
S	8
A	7
B	6
C	2
D	6
E	3
F	1
G	0

4. A* Calculation

Step 1

Start at S:

$$g(S)=0, f(S)=0+8=8$$

From S:

- A: $g=2, f=2+7=9$
- D: $g=6, f=6+6=12$

Choose **A**.

Step 2

From A:

- B: $g=6, f=6+6=12$
- D: $g=8, f=8+6=14 \rightarrow$ existing D path is better
- E: $g=6, f=6+3=9$

Choose **E**.

Step 3

From E:

- B through E gives $g=8 \rightarrow$ worse than B's $g=6$

- D through E gives $g=9 \rightarrow$ worse than D's $g=6$
 - F: $g=10, f=10+1=11$
- Continue with the lowest available evaluation.

Step 4

From B:

- C: $g=9, f=9+2=11$
- G: $g=12, f=12$

Choose C.

Step 5

From C:

$$g(G)=9+2=11$$

$$f(G)=11+0=11$$

Goal reached.

5. Evaluation Table

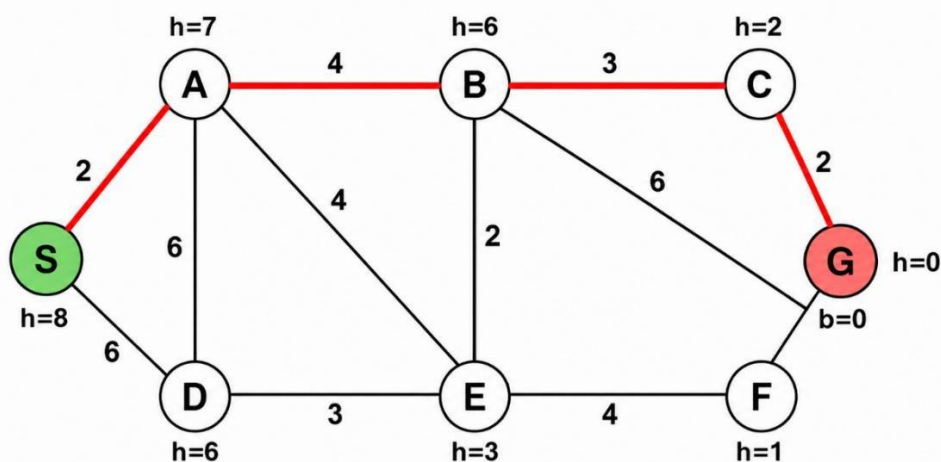
Node	$g(n)$	$h(n)$	$f(n)$
S	0	8	8
A	2	7	9
E	6	3	9
F	10	1	11
B	6	6	12
C	9	2	11
G	11	0	11

6. Optimal Route

The best route is:

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$

Total cost: 11



Optimal Path: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$

Total Cost = $2 + 4 + 3 + 2 = 11$

7. Pseudocode

$A^*(\text{Start}, \text{Goal})$

Put Start in OPEN

$g(\text{Start}) = 0$

while OPEN is not empty:

 select node with lowest $f(n)$

 if node = Goal:

 return path

 expand node

 calculate $g(n)$, $h(n)$, and $f(n)$

 add/update nodes in OPEN

8. Complexity Analysis

- **Time:** $O(b*d)$

- **Space:** $O(b*d)$

A^* is efficient because the heuristic helps guide the search toward the goal.

9. Conclusion

A^* selects: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$

with a minimum cost of: 11

The route is optimal when the given heuristic is admissible.

Q2. Autonomous Warehouse Robot

The question describes a robot whose obstacles are initially unknown and asks for an online intelligent search strategy, obstacle reaction, and comparison with offline search.

1. Problem Analysis

The robot has:

- **Start:** $S=(1,5)$
- **Goal:** $G = (7,1)$
- Grid size: $7*5$
- Unknown cells: ?
- Obstacles: #

The important characteristic is Robot does not know the complete environment initially
Therefore, this is an **online search problem**.

2. Algorithm Selection

Selected Algorithm: Online A* Search

Online A* is suitable because the robot:

1. Observes its surroundings.
2. Updates its map.
3. Calculates a new path.
4. Moves to the next location.
5. Replans whenever an obstacle is discovered.

The assessment itself identifies *Online A Search** as the appropriate strategy.

3. Intelligent Agent Decision Process

Observe → Update Map → Search → Move → Observe Again

If a new obstacle is detected:

Detect Obstacle → Remove Cell → Replan

4. Step-by-Step Execution

Step 1

Robot starts at:

$(1,5)$

Move right:

$(1,5) \rightarrow (2,5)$

Step 2

At $(2,5)$, the robot detects an obstacle at:

$(2,4)$

Therefore, (2,4) is marked as blocked.

The robot moves:

$(2,5) \rightarrow (3,5)$

Step 3

The robot moves downward:

$(3,5) \rightarrow (3,4)$

Step 4

The robot continues:

$(3,4) \rightarrow (3,3)$

Step 5

The robot discovers new obstacles around the current location.

It updates its map and avoids the blocked cells.

The robot replans and moves:

$(3,3) \rightarrow (3,2)$

Step 6

The robot moves:

$(3,2) \rightarrow (4,2)$

Step 7

The robot moves:

$(4,2) \rightarrow (5,2)$

Step 8

At (5,2), the robot detects an obstacle at:

(6,2)

Therefore, it cannot move to (6,2).

The robot replans and moves:

$(5,2) \rightarrow (5,1)$

Step 9

The robot moves:

$(5,1) \rightarrow (6,1)$

Step 10

The robot moves:

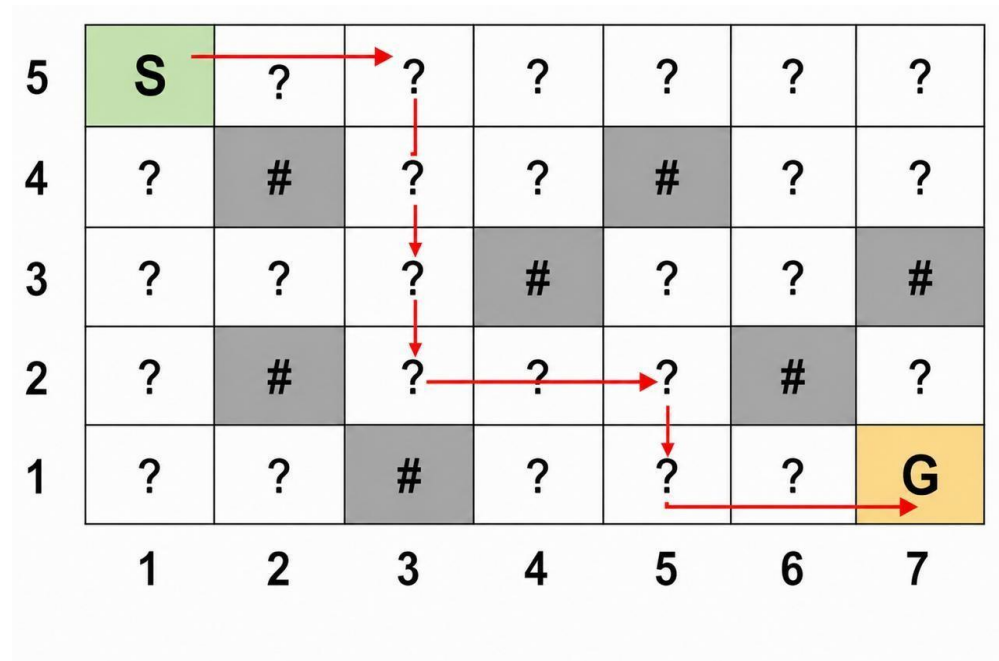
$(6,1) \rightarrow (7,1)$

The target is reached.

4. Final Path

$(1,5) \rightarrow (2,5) \rightarrow (3,5) \rightarrow (3,4) \rightarrow (3,3) \rightarrow (3,2) \rightarrow (4,2) \rightarrow (5,2) \rightarrow (5,1) \rightarrow (6,1) \rightarrow (7,1)$

Total movements = 10



5. Robot's Decision Process

Step	Position	Decision
1	(1,5)	Start
2	(2,5)	Move right
3	(3,5)	Move right
4	(3,4)	Move down
5	(3,3)	Move down
6	(3,2)	Move down
7	(4,2)	Avoid obstacle at (2,2)
8	(5,2)	Move right
9	(5,1)	Avoid obstacle at (6,2)
10	(6,1)	Move right
11	(7,1)	Goal reached

6. Reaction to a New Obstacle

Suppose the robot is planning: $A \rightarrow B \rightarrow C$

and discovers that C is an obstacle.

It performs:

Obstacle Detection → **Map Update** → **Remove Invalid Action** → **Recalculate** → **Replan**

This is the key advantage of online search.

7. Online vs Offline Search

Online Search	Offline Search
Obstacles discovered during movement	Complete map known initially
Replans dynamically	Plans before movement
Suitable for unknown environments	Suitable for known environments
Adapts to new obstacles	Cannot directly handle unknown obstacles

This comparison agrees with the assessment's intended solution.

8. Pseudocode

Online-A*(Start, Goal)

1. Start at S
 2. Observe nearby cells
 3. Update the known map
 4. Generate possible moves
 5. Apply A* to select the best move
 6. Move to the selected cell
 7. If a new obstacle is detected:
 - Update the map
 - Recalculate the path
 8. Repeat until Goal is reached
-

Q3. University Examination Scheduling – CSP

1. CSP Formulation

Variables:

E1, E2, E3, E4, E5, E6

Domains:

Each examination can be assigned to:

{T1, T2, T3, T4}

Objective: Schedule all examinations without student conflicts and while satisfying hall capacities.

2. Constraints

The examinations with common students must be in different time slots:

$E1 \neq E2$

$E1 \neq E3$

$E2 \neq E4$

$E3 \neq E5$

$E4 \neq E6$

$E2 \neq E5$

Hall capacities:

- Hall A = 120
- Hall B = 90
- Hall C = 60

Student numbers:

$E1=100, E2=80, E3=70, E4=60, E5=90, E6=50.$

3. MRV and Forward Checking

Step 1:

Assign:

$E1 = T1, \text{ Hall A}$

Remove T1 from E2 and E3.

Step 2:

Assign:

$E2 = T2, \text{ Hall B}$

Remove T2 from E4 and E5.

Step 3:

Assign:

$E3 = T2, \text{ Hall A}$

Remove T2 from E5.

Step 4:

Assign:

E4 = T1, Hall C

Remove T1 from E6.

Step 5:

Assign:

E5 = T1, Hall B

All conflicts are satisfied.

Step 6:

Assign:

E6 = T2, Hall C

All constraints are satisfied.

4. Domain Updates

Step	Assignment	Domain Update
1	E1=T1	E2,E3 remove T1
2	E2=T2	E4,E5 remove T2
3	E3=T2	E5 removes T2
4	E4=T1	E6 removes T1
5	E5=T1	Valid
6	E6=T2	Valid

5. Backtracking

No backtracking is required because every assignment has at least one valid remaining value.

If a domain becomes empty, the algorithm goes back to the previous assignment and selects another value.

6. Final Timetable

Time	Exam	Hall
T1	E1	A
T1	E4	C
T1	E5	B
T2	E2	B
T2	E3	A
T2	E6	C

Verification

All student conflicts are satisfied:

✓ $E1 \neq E2$

✓ $E1 \neq E3$

✓ $E2 \neq E4$

✓ $E3 \neq E5$

✓ $E4 \neq E6$

✓ $E2 \neq E5$

All hall capacities are satisfied:

✓ Hall A: 100/120, 70/120

✓ Hall B: 90/90, 80/90

✓ Hall C: 60/60, 50/60

Q4. GAME-PLAYING AI – MINIMAX AND ALPHA-BETA PRUNING

1. Problem Analysis

The given game tree has:

- Root node = MAX
- Second level = MIN
- Third level = MAX
- Bottom level = terminal utility values

The **MAX player** tries to get the highest value, while the **MIN player** tries to get the lowest value.

Therefore, the suitable AI technique is **Minimax Search with Alpha-Beta Pruning**.

2. Algorithm Selection

Minimax

Minimax is used to select the best move by considering the opponent's best response.

- MAX → selects maximum value.
- MIN → selects minimum value.

Alpha-Beta Pruning

Alpha-Beta pruning removes branches that cannot affect the final decision.

It gives the same result as Minimax but reduces the number of nodes evaluated.

3. Step-by-Step Minimax Calculation

Step 1: Left MIN Branch

There are three MAX nodes.

First MAX node

Values:

3, 5, 6

MAX selects:

$\max(3, 5, 6) = 6$

Second MAX node

Values:

9, 1, 2

MAX selects:

$\max(9, 1, 2) = 9$

Third MAX node

Values:

0, -1, 4

MAX selects:

$$\max(0, -1, 4) = 4$$

Now MIN selects the minimum:

$$\min(6, 9, 4) = 4$$

Therefore:

$$\text{Left MIN value} = 4$$

4. Middle MIN Branch

First MAX node

Values:

$$7, 5, 8$$

$$\max(7, 5, 8) = 8$$

Second MAX node

Values:

$$6, 2, 3$$

$$\max(6, 2, 3) = 6$$

Third MAX node

Values:

$$4, 0, -2$$

$$\max(4, 0, -2) = 4$$

MIN selects:

$$\min(8, 6, 4) = 4$$

Therefore:

$$\text{Middle MIN value} = 4$$

5. Right MIN Branch

First MAX node

Values:

$$1, 2, 3$$

$$\max(1, 2, 3) = 3$$

Second MAX node

Values:

$$5, -1, 2$$

$$\max(5, -1, 2) = 5$$

Third MAX node

Values:

$$6, 4, 7$$

$$\max(6, 4, 7) = 7$$

MIN selects:

$$\min(3, 5, 7) = 3$$

Therefore:

Right MIN value = 3

6. Final Minimax Decision

The three values received by the root MAX node are:

Left = 4

Middle = 4

Right = 3

MAX selects:

$$\max(4, 4, 3) = 4$$

Therefore:

Minimax value = 4

The AI can select either the **left branch or middle branch**.

7. Alpha-Beta Pruning

Initial values:

Alpha = $-\infty$

Beta = $+\infty$

The tree is evaluated from **left to right**.

Step 1: Left Branch

The left MIN branch gives:

4

Therefore, at the root:

Alpha = 4

Step 2: Middle Branch

The middle MIN branch gives:

4

Therefore:

Alpha = 4

No major branch is pruned in this section.

Step 3: Right Branch

The first MAX node contains:

1, 2, 3

MAX selects:

3

Therefore, at the right MIN node:

Beta = 3

At the root:

Alpha = 4

Now:

$\text{Beta} \leq \text{Alpha}$

$3 \leq 4$

Therefore, the remaining two MAX branches cannot improve the result.

So the following branches are pruned:

(5, -1, 2)

(6, 4, 7)

8. Alpha-Beta Result

Before pruning:

Right MIN branch:

$\min(3, 5, 7) = 3$

After pruning:

Only the first MAX node needs to be evaluated:

$\max(1, 2, 3) = 3$

The other two branches are not evaluated.

Therefore, the final result remains:

Minimax value = 4

9. Minimax Pseudocode

MINIMAX(node):

If node is a terminal node:

 Return its value

If node is MAX:

 Return the maximum value of its children

If node is MIN:

 Return the minimum value of its children

10. Alpha-Beta Pseudocode

ALPHA-BETA(node, alpha, beta):

If node is terminal:

 Return its value

If node is MAX:

 For each child:

 value = ALPHA-BETA(child, alpha, beta)

 alpha = max(alpha, value)

 If alpha $\geq$ beta:

 Stop evaluating remaining children

 Return alpha

If node is MIN:

For each child:

value = ALPHA-BETA(child, alpha, beta)

beta = min(beta, value)

If beta <= alpha:

Stop evaluating remaining children

Return beta

11. Efficiency

Minimax time complexity:

$O(b^d)$

Alpha-Beta pruning with good node ordering can reduce the effective complexity to approximately:

$O(b^{(d/2)})$

Therefore, Alpha-Beta pruning makes the game-playing AI faster by avoiding unnecessary branches.

12. Final Answer

Left MIN = 4

Middle MIN = 4

Right MIN = 3

Therefore:

MAX = max(4, 4, 3) = 4

Final Minimax Value: 4

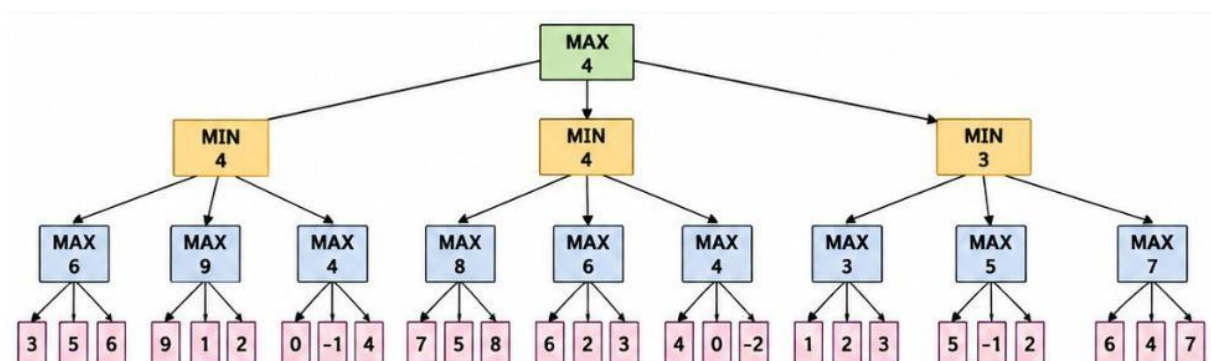
Best Move:

Left or Middle branch

Alpha-Beta Pruned Branches:

(5, -1, 2)

(6, 4, 7)



Conclusion:

The AI selects a move with a Minimax value of 4. Alpha-Beta pruning removes branches

that cannot affect the final decision, reducing computation while producing the same optimal result.

Q5. DELIVERY ROUTE OPTIMIZATION – HILL CLIMBING

1. Problem Analysis

The objective is to minimize the total distance travelled by the delivery vehicle.

This is a **Local Search Optimization** problem. Since the question provides an initial route and neighbouring routes, **Hill Climbing** is suitable.

2. Algorithm Selected

The selected algorithm is:

Hill Climbing

For this minimization problem:

Smaller distance = Better solution

The algorithm always moves from the current route to a neighboring route with a lower cost.

3. Given Neighbouring Routes

The given routes and their costs are:

S1: D → A → C → B → E → D = 26 km

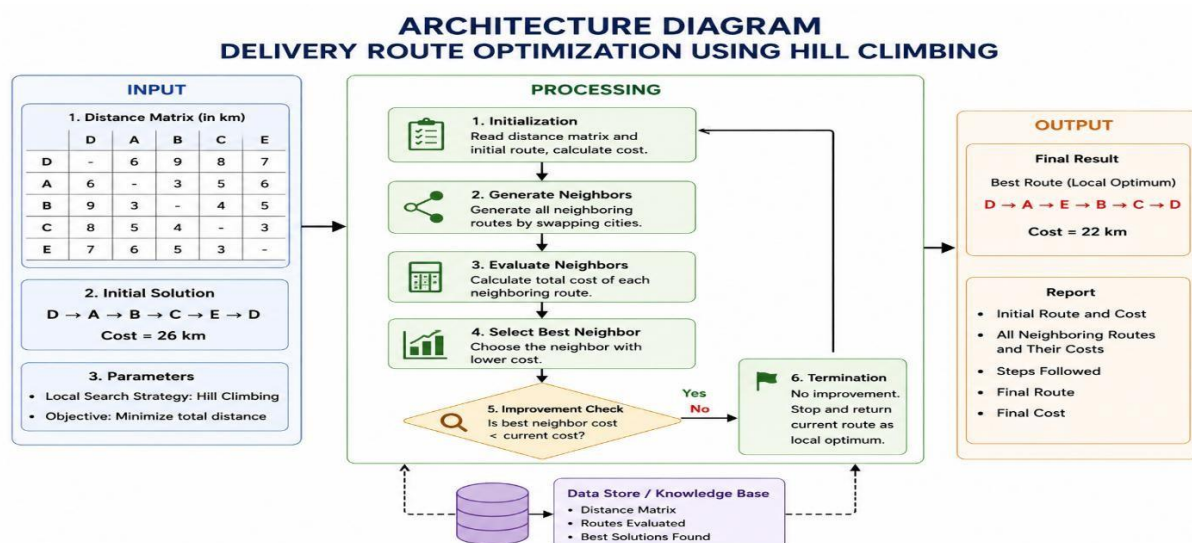
S2: D → B → A → C → E → D = 30 km

S3: D → A → B → E → C → D = 24 km

S4: D → E → C → B → A → D = 27 km

S5: D → A → E → B → C → D = 23 km

S6: D → C → B → A → E → D = 26 km



4. Step-by-Step Hill Climbing

Step 1: Compare all neighbouring routes

Costs are:

26, 30, 24, 27, 23, 26

The lowest cost is:

23 km

Therefore, select:

S5 = D → A → E → B → C → D

Current cost = **23 km**

Step 2: Check the neighbours of S5

The question states that the best neighbouring solution from S5 has a cost of:

22 km

Since:

$22 < 23$

the algorithm moves to the 22-km solution.

Step 3: Check the new solution

The question states that all neighbouring solutions of the 22-km solution have costs:

≥ 22 km

Therefore, there is no better neighbouring solution.

Step 4: Stop

Hill Climbing stops because no neighbouring solution has a smaller cost.

Therefore:

Final Local Optimum = 22 km

5. Is It a Global Optimum?

The 22-km solution is a **local optimum**, because none of its neighbouring solutions is better.

It is **not guaranteed to be a global optimum**, because Hill Climbing checks only the current neighbourhood.

6. How to Escape Local Optimum

We can use:

1. **Random Restart Hill Climbing** – Start from a different initial route.
2. **Simulated Annealing** – Sometimes accept a worse solution to escape the local optimum.
3. **Stochastic Hill Climbing** – Select randomly from good neighbouring solutions.

7. Hill Climbing Procedure

1. Select the initial route.
2. Generate neighbouring routes.

3. Calculate the cost of each route.
4. Select the route with the lowest cost.
5. Move to the better neighbour.
6. Repeat the process.
7. If no neighbour is better, stop.
8. The current solution is a local optimum.

8. Pseudocode

Hill-Climbing(Route)

1. current = Route
2. Generate neighbouring routes
3. Select the neighbour with minimum cost
4. If neighbour cost < current cost:
 current = neighbour
 Repeat
5. Else:
 Stop
6. Return current

9. Final Answer

Best given route: S5

Initial cost: 23 km

Improved neighbouring solution: 22 km

Final local optimum: 22 km

Global optimum: Not guaranteed

Method to escape local optimum: Random Restart or Simulated Annealing

Conclusion

Hill Climbing improves the delivery route by repeatedly selecting a better neighbouring route. In the given problem, it reaches a **local optimum of 22 km**. Since Hill Climbing may get trapped in a local optimum, **Random Restart or Simulated Annealing** can be used to search for a better global solution.

EVALUATION RUBRICS

Criteria	Weightage (%)		Excellent (Full Marks)	Good	Average	Poor
CSP Problem Formulation			0% 4% – arly defines all variables, domains, and problem objectives correctly.	3% – Defines most variables and domains with minor errors.	2% – Partial formulati on with some missing components.	0–1% – Incorrect or incomple te formulation.
Constraint Identificati on & Representat io n			4% – Accurately identifies all student-confli ct and hall-capacity constraints and represents them correctly.	3% – Identifies most constraints with minor errors.	2% – Identifies some constraints but misses important relationships .	0–1% – Constraints arelargely incorrect or missing.
Application of Backtrackin g, MRV & Forward Checking			6% – Correctly applies Backtracki ng, MRV, and Forward Checking withclear and logical steps.	4.5% – Applies the technique s correctly with minor mistakes.	3% – Partial applicatio n with unclear or inconsistent steps.	0–1.5% – Incorrect or inappropria te application of CSP techniques.

Step-by-Step Execution & Domain Updates			20% 4% – Provides a complete, logical execution trace and correctly updates domains after every assignment.	3% – Mostly correct execution with minor gaps.	2% – Provides limited steps with some incorrect domain updates.	0–1% – Incomplete or incorrect execution.
---	--	--	---	--	---	---

Final Timetable, Accuracy & Justification		2% – Produces a valid conflict-free timetable satisfying all constraints and capacity requirements with clear justification.	1.5% – Valid timetable with minor errors in justification.	1% – Partially valid timetable with some constraint violations.	0–0.5% – Invalid or missing timetable.
---	--	--	--	---	--